

Project Specification: RubbleSim

1. Project Name

RubbleSim

Subtitle: A real-time C++ debris interaction simulator for robot navigation, contact visualization, deterministic replay, profiling, and synthetic sensor-map export.

2. One-Line Description

RubbleSim is a real-time C++ simulation and rendering sandbox where a robot blade or rover interacts with unstable debris piles, clearing blocked paths while the system visualizes contacts, object motion, collision graphs, occupancy maps, depth maps, and performance telemetry.

3. Core Idea

Build an interactive C++ simulation where a simple robot-like pusher, blade, or rover moves through cluttered debris and physically interacts with unstable objects.

The scene should include:

- rubble chunks,
- boxes,
- pipes,
- planks,
- barriers,
- unstable stacks,
- blocked passages,
- a robot pusher/blade,
- multiple camera views,
- debug overlays.

The simulator should support:

- object-object collisions,
- robot-object interactions,
- debris shifting,
- pile collapse,
- path clearing,
- contact visualization,
- deterministic replay,

- benchmark telemetry,
- synthetic sensor exports.

The central idea:

Simulate robot interaction with cluttered, unstable debris in real time, while exposing the kind of debugging, profiling, and data outputs expected from simulation/graphics infrastructure.

This project should feel like a small but serious robotics simulation engine demo, not a Python robotics toy and not a copied excavation simulator.

4. Why This Project Matters

Robots operating in real environments often face messy physical obstacles: disaster rubble, warehouse spills, cluttered corridors, industrial debris, fallen objects, and blocked paths.

Simulation and graphics-infra teams need engineers who can:

- work in C++,
- structure real-time simulation systems,
- implement collision and contact logic,
- build debug visualization tools,
- profile simulation and rendering bottlenecks,
- export useful spatial data,
- create deterministic replay systems,
- connect simulation to synthetic perception outputs.

RubbleSim demonstrates:

- C++ simulation engineering,
- OpenGL rendering,
- custom collision broadphase,
- contact and motion visualization,
- real-time profiling,
- deterministic replay,
- sensor-map export,
- robotics-oriented simulation tooling.

The project is designed to be visually memorable while still giving a strong infrastructure signal.

5. Target Portfolio Signal

This project should signal:

"I can build real-time C++ simulation and rendering infrastructure for robotics-relevant physical environments, including collision/contact systems, debug visualization, deterministic replay, profiling, and synthetic sensor export."

It should fit into the broader portfolio as the **C++ graphics/simulation infrastructure project**.

It is not primarily a machine learning project.

It is not primarily a robot policy project.

It is not primarily a dataset-generation project.

It is the project that proves:

"I can work close to the engine."

6. Target Roles

This project is designed to support applications for:

- simulation infrastructure roles,
- graphics infrastructure roles,
- robotics simulation roles,
- NVIDIA Omniverse / Isaac-style teams,
- Unity/Unreal physics and tools roles,
- robotics simulator companies,
- synthetic data infrastructure teams,
- C++ graphics/simulation engineering internships.

It is especially relevant for roles where the signal is:

"I can work inside a real-time simulation/rendering codebase, extend the physics/debug/data pipeline, and profile the system."

7. Tech Stack

Core Language

- C++17 or C++20

Build System

- CMake

Rendering

Preferred:

- OpenGL
- GLFW
- GLAD
- GLM
- ImGui

Optional:

- tinyobjloader for simple mesh loading
- stb_image / stb_image_write for image export

Simulation

Custom lightweight simulation layer.

Implement enough logic to show infrastructure skill:

- rigid object transforms,
- AABB collision,
- optional OBB collision,
- spatial hash grid broadphase,
- simple impulse/push response,
- friction/damping approximation,
- sleeping/settling logic,
- contact graph logging.

Do not build a full production physics engine.

Data Export

- JSON metadata
- CSV benchmark logs
- PNG screenshots
- PGM/PNG depth or height maps
- object ID masks
- occupancy maps
- contact heatmaps
- replay logs

Debug / Profiling

- ImGui overlay
- CPU timers
- frame-time graph

- simulation step timing
- collision broadphase timing
- narrowphase/contact timing
- render timing
- active object count
- contact pair count
- p50/p95 frame time

Repo Tooling

- clang-format
 - simple unit tests if time allows
 - GitHub Actions optional
 - README with demo GIFs, architecture diagram, benchmark tables, and limitations
-

8. What We Are Building

Build a real-time C++ debris simulation system with six main parts.

Part A — Debris Arena Scene

Create a small robotics test arena.

The scene should include:

- floor plane,
- walls or barriers,
- blocked doorway,
- scattered debris,
- unstable stack,
- robot blade or pusher,
- camera controls,
- debug draw system.

Debris Object Types

Start simple:

- boxes,
- rectangular planks,
- cylinders/pipes,
- irregular rubble chunks approximated by boxes or convex primitives,
- barriers,
- crates.

Required Scene Variants

Build at least three:

1. **Blocked Doorway**

2. Debris blocks a passage.
3. Robot blade pushes objects aside.
4. Goal is to expose a traversable path.

5. **Unstable Stack Collapse**

6. A pile or stack is hit by the robot.
7. Objects collapse and scatter.
8. Goal is to show chain reactions and contact visualization.

9. **Narrow Path Clearing**

10. Robot must push through clutter in a corridor.
11. Objects shift, jam, or clear.
12. Goal is to show occupancy map changes over time.

Optional fourth:

1. **Pipe and Plank Jam**

2. Long objects jam against each other.
3. Useful for showing contact graph and collision debugging.

Part B — Simulation Core

Implement a simple but serious real-time simulation layer.

Object State

Each object should store:

```
object_id
shape_type
position
orientation
linear_velocity
angular_velocity optional
mass
friction
restitution
is_static
is_sleeping
bounding_volume
```

Robot Tool State

The robot does not need to be a full robot.

V1 can be:

- a controllable blade,
- a box-shaped pusher,
- a wedge,
- a simple rover body with a front blade.

The robot tool should support:

- keyboard movement,
- scripted movement,
- replayed movement,
- collision with debris,
- force/contact visualization.

Physics Features

Required:

- gravity optional depending on scene,
- AABB broadphase,
- simple collision detection,
- contact point approximation,
- push/impulse response,
- friction/damping,
- object settling,
- sleeping objects,
- deterministic timestep.

Optional:

- OBB collision,
- cylinder approximation,
- angular response,
- stack stability heuristic,
- collision layers,
- kinematic robot body.

Broadphase

Implement a spatial hash grid.

Track:

- object count,
- occupied grid cells,
- candidate collision pairs,
- actual contact pairs,
- broadphase time.

This is important because it gives the project real simulation-infra substance.

Contact Graph

At each timestep, log:

```
frame_id
object_a
object_b
contact_position
contact_normal
relative_velocity
contact_strength_proxy
```

The contact graph should be visualized in the debug view.

Part C — OpenGL Rendering

Build a real-time renderer for the arena.

Rendered Elements

- debris objects,

- robot blade/pusher,
- floor grid,
- walls/barriers,
- debug bounding boxes,
- contact points,
- contact normals,
- velocity arrows,
- occupied grid cells,
- camera frustum,
- cleared path overlay.

Visual Style

The demo should be clean and readable, not photorealistic.

Target visual style:

- robotics lab,
- disaster-response test arena,
- clean geometry,
- high-contrast debug overlays,
- readable object colors,
- clear camera movement.

Required Views

1. Main Render View

2. Normal shaded scene.

3. Debug Contact View

4. Contact points, normals, object IDs, bounding boxes.

5. Top-Down Occupancy View

6. Shows blocked/free space.

7. Sensor View

8. Simulated depth/object-ID/contact map from a fixed camera.

Part D — Replay System

Build deterministic replay.

This is a major credibility feature.

Replay Inputs

Save:

```
scenario_seed
initial_object_states
robot_control_inputs
timestep
simulation_config
```

Replay Outputs

Allow:

- replay exact scenario,
- pause/play,
- scrub timeline,
- switch camera view,
- toggle debug overlays,
- export selected frames,
- compare before/after occupancy maps.

Why Replay Matters

Replay makes the project feel like real infrastructure.

It shows that the simulator is not just a visual toy. It can reproduce bugs, analyze collisions, and generate consistent outputs.

Part E — Benchmark and Profiling Mode

Add a benchmark mode from the beginning.

Metrics to Track

Per frame:

```
frame_id
total_frame_time_ms
simulation_time_ms
broadphase_time_ms
```

```
contact_time_ms  
render_time_ms  
active_objects  
candidate_pairs  
contact_pairs  
dirty_objects  
draw_calls
```

Aggregate:

```
average FPS  
p50 frame time  
p95 frame time  
max frame time  
average broadphase time  
average render time  
average contact pairs
```

Benchmark Scenarios

Run fixed benchmark scenes:

1. 50 objects
2. 100 objects
3. 250 objects
4. 500 objects if possible

Goal is not to beat a real engine.

Goal is to show honest profiling discipline.

Part F — Synthetic Sensor and Map Export

Export robot-relevant spatial data from the simulation.

Required Exports

For selected frames:

```
rgb/frame_0000.png  
depth/frame_0000.png or .npy  
object_id_mask/frame_0000.png  
occupancy/frame_0000.png
```

```
contact_heatmap/frame_0000.png
poses/frame_0000.json
contacts/frame_0000.json
```

Occupancy Map

Generate a top-down grid:

- free,
- occupied,
- robot,
- cleared path,
- unknown optional.

Contact Heatmap

Generate a top-down heatmap showing where contact activity happened.

This gives the project a nice robotics/data-pipeline connection without turning it into a Python perception project.

9. Demo Requirements

The final demo should be visually strong and immediately understandable.

The viewer should understand:

“A robot is interacting with unstable debris, the system simulates contacts in real time, and the engine exposes debug, replay, profiling, and sensor outputs.”

Demo View 1 — Blocked Doorway Clearing

Show a robot blade pushing debris away from a blocked passage.

Display:

- normal rendered scene,
 - object movement,
 - path before/after,
 - top-down occupancy map.
-

Demo View 2 — Contact Debug Overlay

Switch to debug mode.

Show:

- bounding boxes,
- contact points,
- contact normals,
- velocity arrows,
- active collision pairs,
- broadphase grid cells.

This is the most important view for simulation-infra signal.

Demo View 3 — Unstable Stack Collapse

Show a chain reaction.

A robot pushes one object, causing a stack to collapse.

Display:

- contact graph,
 - object trajectories,
 - collision events,
 - frame-time overlay.
-

Demo View 4 — Deterministic Replay

Replay the same scenario.

Show:

- same seed,
 - same robot commands,
 - same object motion,
 - scrub timeline,
 - camera switch,
 - export frame.
-

Demo View 5 — Benchmark Mode

Show fixed benchmark scenes.

Display:

- object count,
 - contact pairs,
 - FPS,
 - p95 frame time,
 - broadphase time,
 - render time.
-

Demo View 6 — Sensor Export Preview

Show output folder or dashboard:

```
rgb/  
depth/  
object_id_mask/  
occupancy/  
contact_heatmap/  
poses/  
contacts/  
benchmark.csv  
replay.json
```

End with a clear claim:

Real-time C++ robot-debris simulation with contact debugging, deterministic replay, profiling, and synthetic map export.

10. Scope

The project should focus on real-time C++ simulation infrastructure, debug visualization, replay, profiling, and export.

It should not become a full physics engine or full robotics simulator.

In Scope

- C++ simulation core,
 - OpenGL rendering,
 - controllable robot blade/pusher,
 - debris objects,
 - broadphase collision grid,
 - simple contact response,
 - contact visualization,
 - debug overlays,
 - deterministic replay,
 - benchmark mode,
 - occupancy map export,
 - object ID mask export,
 - depth map export,
 - contact heatmap export,
 - final demo video,
 - clean README and technical report.
-

Out of Scope

Do not build:

- full PhysX replacement,
 - full rigid-body engine,
 - deformable soft-body simulation,
 - fluid simulation,
 - photorealistic rendering,
 - full robot kinematics,
 - robot policy learning,
 - reinforcement learning,
 - neural world model,
 - SLAM,
 - path planning stack,
 - ROS2 integration in V1,
 - Unity/Unreal integration,
 - large asset library,
 - complicated mesh fracture,
 - CUDA physics solver,
 - differentiable simulation.
-

11. Project Size

Target size:

- medium-to-large C++ graphics/simulation systems project,
- visual-demo focused,
- roughly 60–100 focused hours,
- 2,500–6,000 lines of code.

Expected repository structure:

```
rubblesim/  
  CMakeLists.txt  
  README.md  
  assets/  
  configs/  
  src/  
    main.cpp  
    app/  
    sim/  
    physics/  
    collision/  
    rendering/  
    replay/  
    export/  
    profiling/  
    ui/  
  include/  
  shaders/  
  scripts/  
  docs/  
  output/  
  tests/
```

12. Intended Timeline

Week 1 — C++ Engine Skeleton + Basic Scene

Goals

- set up CMake project,
- create window with GLFW/OpenGL,
- add camera controls,
- render floor/grid,

- render boxes/cylinders as simple primitives,
- add ImGui overlay,
- create basic scene loader/config,
- add simple robot pusher object.

Deliverable

A real-time C++ app that renders a debris arena and allows a robot pusher to move around.

Week 2 — Collision + Contact System

Goals

- implement object state system,
- implement AABB collision,
- implement spatial hash broadphase,
- implement simple contact response,
- add friction/damping,
- add sleeping/settling,
- visualize bounding boxes,
- visualize contact points and normals,
- log contact pairs.

Deliverable

A debris scene where the robot pusher can move objects, trigger collisions, and show debug contact visualization.

Week 3 — Scenarios + Replay

Goals

- build Blocked Doorway scenario,
- build Unstable Stack Collapse scenario,
- build Narrow Path Clearing scenario,
- save scenario seed,
- save robot input sequence,
- implement deterministic replay,
- add timeline scrubber,
- add camera switching,
- export replay video or image sequence.

Deliverable

A replayable simulator with three visually distinct robot-debris interaction scenarios.

Week 4 — Profiling + Sensor Export + Polish

Goals

- add benchmark mode,
- log frame-time metrics,
- log broadphase/contact/render timing,
- generate occupancy maps,
- generate object ID masks,
- generate depth maps,
- generate contact heatmaps,
- export object pose JSON,
- create final demo video,
- write README,
- write technical report,
- add architecture diagram,
- add limitations section.

Deliverable

A resume-ready C++ simulation/graphics infrastructure project with demo GIFs, benchmark tables, replay files, exported sensor maps, and clear documentation.

13. Success Criteria

The project is successful if it can honestly claim:

RubbleSim is a real-time C++ simulation and rendering sandbox for robot-debris interaction, with custom broadphase collision, contact visualization, deterministic replay, benchmark telemetry, and synthetic map export.

Concrete success targets:

Metric	Target
Language	C++17 or C++20
Rendering	OpenGL real-time viewer
Scenario types	3
Objects per scene	50-250
Collision broadphase	spatial hash grid
Debug overlays	contacts, normals, bounding boxes, grid cells
Replay	deterministic seed + input replay
Benchmark mode	included
Exported maps	RGB, depth, object ID, occupancy, contact heatmap
Performance metrics	FPS, p50/p95 frame time, sim/render/contact time

Final demo
README
limitations

60–90 seconds
demo GIF, architecture, benchmark table,

14. Evaluation Plan

This project is not evaluated by ML accuracy.

It should be evaluated by:

- simulation correctness,
- real-time performance,
- debug visibility,
- replay determinism,
- export usefulness,
- code organization.

Simulation Metrics

Measure:

- number of active objects,
- number of candidate collision pairs,
- number of actual contact pairs,
- object displacement,
- path clearance percentage,
- object settling time,
- contact graph density.

Rendering / Performance Metrics

Measure:

- FPS,
- total frame time,
- p50 frame time,
- p95 frame time,
- simulation step time,
- broadphase time,
- contact response time,

- render time,
- draw calls.

Export Metrics

Measure:

- RGB export time,
- depth export time,
- occupancy map export time,
- contact heatmap export time,
- replay file size,
- benchmark log size.

Required Tables

Table 1 — Scenario Summary

Scenario	Objects	Contact Pairs	Settling Time	Path Cleared %
Blocked Doorway				
Stack Collapse				
Narrow Path				
Pipe/Plank Jam				

Table 2 — Performance Summary

Objects	FPS Avg	Frame p50	Frame p95	Sim ms	Broadphase ms	Render ms
50						
100						
250						
500						

Table 3 — Export Summary

Output Type	Resolution/Grid Size	Export Time	File Size
RGB			
Depth			
Object ID Mask			
Occupancy Map			
Contact Heatmap			

15. What To Do

Do:

- build the C++ app skeleton first,
 - make the demo visual early,
 - implement contact visualization early,
 - keep physics simple but honest,
 - prioritize deterministic replay,
 - build benchmark mode before polishing visuals,
 - use ImGui for debug controls,
 - show timing metrics in the UI,
 - make scene configs reproducible,
 - keep the robot simple,
 - write clean architecture docs,
 - include limitations honestly.
-

16. What Not To Do

Do not:

- build a full physics engine,
 - copy the excavation terrain theme,
 - spend weeks on asset quality,
 - implement complex fracture,
 - add neural networks,
 - add RL,
 - add robot learning,
 - add ROS2 in V1,
 - make the simulation physically perfect,
 - hide that the physics is simplified,
 - overbuild menus/UI before simulation works,
 - skip profiling,
 - skip replay,
 - skip debug overlays,
 - skip benchmark tables.
-

17. Repository Structure

Recommended structure:

```
rubblesim/  
  README.md  
  CMakeLists.txt  
  LICENSE  
  .clang-format  
  
  configs/  
    blocked_doorway.json  
    stack_collapse.json  
    narrow_path.json  
    benchmark_50.json  
    benchmark_100.json  
    benchmark_250.json  
  
  assets/  
    textures/  
    meshes/  
  
  shaders/  
    basic.vert  
    basic.frag  
    debug.vert  
    debug.frag  
    depth.vert  
    depth.frag  
  
  include/  
    rubblesim/  
  
  src/  
    main.cpp  
  
  app/  
    Application.cpp  
    Application.h  
    Input.cpp  
    Input.h  
    CameraController.cpp  
    CameraController.h  
  
  sim/  
    World.cpp
```

```
World.h
Object.cpp
Object.h
RobotPusher.cpp
RobotPusher.h
ScenarioLoader.cpp
ScenarioLoader.h

collision/
  AABB.cpp
  AABB.h
  SpatialHashGrid.cpp
  SpatialHashGrid.h
  CollisionSystem.cpp
  CollisionSystem.h
  Contact.cpp
  Contact.h

physics/
  Integrator.cpp
  Integrator.h
  ContactSolver.cpp
  ContactSolver.h

rendering/
  Renderer.cpp
  Renderer.h
  Mesh.cpp
  Mesh.h
  PrimitiveFactory.cpp
  PrimitiveFactory.h
  DebugDraw.cpp
  DebugDraw.h
  Framebuffer.cpp
  Framebuffer.h

replay/
  ReplayRecorder.cpp
  ReplayRecorder.h
  ReplayPlayer.cpp
  ReplayPlayer.h

export/
  ImageExporter.cpp
  ImageExporter.h
  MapExporter.cpp
  MapExporter.h
  JsonExporter.cpp
```

```
    JsonExporter.h

profiling/
  Profiler.cpp
  Profiler.h
  Benchmark.cpp
  Benchmark.h

ui/
  DebugPanel.cpp
  DebugPanel.h

scripts/
  build.sh
  run_demo.sh
  run_benchmark.sh
  export_demo_frames.sh

docs/
  architecture.md
  collision_system.md
  replay_system.md
  results.md
  limitations.md

output/
  replays/
  screenshots/
  maps/
  benchmarks/

tests/
  test_spatial_hash.cpp
  test_aabb.cpp
  test_replay_determinism.cpp
```

18. README Requirements

The README should include:

1. project title and one-line description,
2. demo GIF at the top,
3. why robot-debris simulation matters,
4. feature list,
5. architecture diagram,

6. simulation system explanation,
 7. collision/contact system explanation,
 8. rendering/debug overlay screenshots,
 9. deterministic replay explanation,
 10. benchmark mode explanation,
 11. exported sensor/map examples,
 12. performance tables,
 13. how to build,
 14. how to run demo scenarios,
 15. how to run benchmark mode,
 16. limitations,
 17. future improvements.
-

19. Dataset / Export Schema

Each generated scenario should follow this structure:

```
output/  
  scenario_0001/  
    metadata.json  
    replay.json  
    benchmark.csv  
  
  frames/  
    rgb/  
      frame_0000.png  
    depth/  
      frame_0000.png  
    object_id/  
      frame_0000.png  
    occupancy/  
      frame_0000.png  
    contact_heatmap/  
      frame_0000.png  
  
  poses/  
    frame_0000.json  
  
  contacts/  
    frame_0000.json  
  
  videos/  
    rendered_replay.mp4
```

```
debug_overlay.mp4
occupancy_replay.mp4
```

Metadata should include:

```
scenario_name
scenario_seed
object_count
object_types
robot_tool_type
simulation_dt
frame_count
camera_params
benchmark_enabled
exported_outputs
```

Replay JSON should include:

```
initial_object_states
robot_input_sequence
scenario_seed
simulation_config
```

Contact JSON should include:

```
frame_id
contact_pairs
object_a
object_b
contact_position
contact_normal
contact_strength_proxy
```

20. Final Demo Video Script

The final demo should be 60–90 seconds.

Suggested structure:

1. Show the RubbleSim title and one-line claim.
2. Show the blocked doorway scenario before the robot acts.

3. Move the robot blade into the debris.
4. Show debris shifting and the path opening.
5. Toggle contact debug overlay.
6. Show contact points, normals, bounding boxes, and broadphase grid.
7. Switch to top-down occupancy view.
8. Show before/after occupancy map.
9. Show unstable stack collapse scenario.
10. Pause and replay the scenario deterministically.
11. Show benchmark overlay with FPS, frame time, broadphase time, render time, and contact pairs.
12. Show exported RGB/depth/object-ID/contact maps.
13. End with benchmark table and final technical claim.

Final statement:

RubbleSim is a real-time C++ simulation and rendering sandbox for robot-debris interaction, with contact debugging, deterministic replay, benchmark telemetry, and synthetic sensor-map export.

21. Final Technical Claim

The final project should support this claim:

Real-time C++ simulation infrastructure can support robotics-relevant debris interaction scenes by combining lightweight collision/contact systems, OpenGL debug rendering, deterministic replay, profiling, and synthetic spatial-map export.

22. Resume Bullet

Full Version

RubbleSim — Built a real-time C++ robot-debris interaction simulator with OpenGL rendering, spatial-hash collision broadphase, contact/debug visualization, deterministic replay, benchmark telemetry, and synthetic depth/occupancy/contact-map export.

Shorter Version

RubbleSim — Built a C++ real-time debris simulation sandbox with OpenGL rendering, contact visualization, deterministic replay, profiling, and synthetic sensor-map export.

More Simulation-Infra Version

RubbleSim — Built a C++ simulation/graphics infrastructure demo for robot-debris interaction, implementing spatial-hash broadphase collision, real-time OpenGL debug rendering, deterministic replay, and benchmark profiling.

More Robotics Version

RubbleSim — Built a robotics-focused debris interaction simulator that visualizes contact dynamics, path clearing, occupancy changes, and synthetic sensor outputs in a real-time C++/OpenGL engine.

23. Portfolio Positioning

RubbleSim should be positioned as the third pillar of the portfolio:

DriveSim Oracle	-> learned world models / counterfactual prediction
Isaac Skill Atlas	-> robotics simulation / policy evaluation
RubbleSim	-> C++ simulation and graphics infrastructure

It should not compete with DriveSim Oracle.

It should not become another Python robotics simulator.

It should clearly own the low-level systems signal:

C++, real-time rendering, collision/contact systems, debug tooling, replay, profiling, export pipeline.

24. Future Improvements

Possible future extensions:

- OBB collision,
- capsule/cylinder collision,
- instanced rendering for many debris objects,
- GPU picking,
- BVH broadphase,
- robot path planner over exported occupancy map,
- Isaac Sim comparison,
- ROS2 bridge,
- CUDA contact-map generation,
- simple learned surrogate predicting path clearance,
- video dataset export for robot perception.

Only add these after the main project is finished.

25. Main Warning

Do not let the project become too mathematically ambitious.

The goal is not:

“I built a perfect physics engine.”

The goal is:

“I built a clean, real-time C++ simulation infrastructure demo with collision, rendering, replay, profiling, and robotics-relevant exports.”

That is the hiring signal.